Python に関するクイックリファレンス

秋山進一郎
筑波大学計算科学研究センター

2026 年 10 月 21 日

目次

1 はじめに 2

2 Python の特徴 3
- 2.1 動的型付き言語 . . . 3
- 2.2 インデントによるコードブロックの表現 . . . 4
- 2.3 豊富なライブラリ . . . 5
- 2.4 Python の予約語 . . . 5
- 2.5 PEP8 . . . 6

3 型 6
- 3.1 数値型 . . . 6
- 3.2 真偽値 . . . 7

4 代表的なコンテナ型 8
- 4.1 リスト . . . 8
- 4.2 リストに対する処理 . . . 9
- 4.3 タプル . . . 10
- 4.4 集合 . . . 11
- 4.5 辞書 . . . 11

5 関数 12

5.1 def 文 12
5.2 ラムダ式 13

6 データの入出力 13

7 ライブラリを import する方法 14

8 import によるスクリプトの分割 15

9 matplotlib による作図 17
9.1 最も基本的な使い方 17
9.2 誤差バー付きのグラフ 19
9.3 散布図 21
9.4 複数のデータのプロット 23
9.5 LaTeX の記法を使う 24
9.6 変数の埋め込み 26

10 仮想環境 28

1 はじめに

例えば，10 行 10 列の二つ行列の積を計算したいとしよう．その計算方法は至ってシンプルであるが，実際に手で計算するのは大変骨が折れる．

$$(AB)_{ij} = \sum_{k=1}^{10} A_{ik} B_{kj} \tag{1.1}$$

式 (1.1) を見れば分かるように，10 行 10 列からなる行列同士の積を計算するためには，式 (1.1) の計算を ij に関して 100 回繰り返さなければならない．仮に，式 (1.1) の右辺で，それぞれの積 $A_{ik}B_{kj}$ を α 秒で，それらの k に関する和を β 秒で計算できたとしよう．計算に要する時間は $(\alpha \times 10 + \beta) \times 10 \times 10$ 秒だ．仮に $\alpha = \beta = 1$ でミスなく計算できた（！）とすると，18 分くらいで答えを得ることができる．もっと一般の行列積を考えて，M 行 K 列の行列と K 行 N 列の行列の積を計算するとしよう．この場合に必要な計算時間は $(\alpha \times K + \beta) \times M \times N$ 秒であり，M, N, K が大きい場合，計算時間は $O(MNK)$ でスケールすることになる．

この種の計算をやらねばならないというニーズが生じた場合，計算機は強力な助っ人に

なる．しかし，計算機は**機械語**という数字の羅列によって記述される言語しか理解することができない．行列積の計算を実行させたいだけなのに，機械語を一から学ばなければならない，と言われたら意気消沈するだろう．このギャップを埋めてくれるのが**プログラミング言語**である．プログラミング言語とは，人間が計算機に指示を与えるための言語であり，人間にとって分かりやすい言語体系になっている．プログラミング言語によって書かれた文章（プログラム）の実行方式は二種類に分けることができる．一つは，人間が書いたプログラムを上から一行ずつ機械語に翻訳しながら計算機に渡していくインタプリ型の言語である．このようなプログラミング言語は**スクリプト言語**と呼ばれ，Python はスクリプト言語の一例である．もう一つは，人間が書いたプログラムを最初から最後まで機械語に翻訳し切った後に計算機に一気に渡すコンパイル方式である．こうしたプログラミング言語は**コンパイル言語**と呼ばれ，C，C++，Fortran などが有名である．

2 Python の特徴

Python の特徴として，ここでは以下の三項目について説明する．

2.1 動的型付き言語

Python に限らず，プログラミング言語には**変数**と**型**という概念がある．変数とは，計算機上で再利用可能なように名前が付けられた“値”（より正確には，メモリ上の番地）のことであり，変数にはいくつかの型が存在する．Python で変数を扱う際には，変数の型を事前に宣言する必要がない．これは，Python が動的型付き言語だからである．具体例を見た方が早いので，まずはソースコード 1 を見てほしい．

ソースコード 1　整数 (`int`) 型

```
1  a = 1
```

これは，「`a` という変数を用意し，そこに `1` という値を代入せよ」という意味になる．この時，変数 `a` の型は**整数 (`int`) 型**になっている．ここで重要な点は，**`a` という変数に `1` という整数値が代入されたことで，`a` の型が整数 (`int`) 型に決まった**ということだ．[*1]

変数に小数点を含む数字を代入すると，その変数は**浮動小数点 (`float`) 型**になる（ソースコード 2）．

[*1] 例えば，C や Fortran では変数の型を事前に宣言しないといけない．

ソースコード 2　浮動小数点 (float) 型

```
1 b = 1.0
```

ダブルクォーテーションあるいはクォーテーションで囲まれた文字が代入された変数は，**文字列（str）型**になる（ソースコード 3）．

ソースコード 3　文字列（str）型

```
1 c = "string"
```

一般的に，動的型付き言語では変数の型宣言が不要なため，少ない記述量でコードが書けるが，実行中の処理速度は他の言語で書いた場合に比べて遅くなることも少なくない．

2.2　インデントによるコードブロックの表現

プログラムはいくつかの**文**から構成される．文はプログラムの実行単位であり，プログラムを実行すると，原則として**上の文から順番にプログラムが実行されていく**．Python では，インデント（行頭の字下げ）によって**コードブロック**を表現する．言い換えると，Python ではプログラム上に書かれた文だけでなく，空白もプログラム上で意味を持つ．具体例として，以下のソースコード 4 を見てみよう．

ソースコード 4　インデントによるコードブロックの表現

```
1 if x > 0 :
2     print("Pro")
3 else:
4     print("Con")
```

Python では，この例のように，「キーワード + コロン（:）」でコードブロックが始まり，続くコードブロックはインデント（行頭の字下げ）で表される．インデントでコードブロックを表現する方法は**オフサイドルール**と呼ばれ，Python の大きな特徴の一つである．インデントは空白何文字でも構わないが，**スペース 4 つ**で入れるのが推奨されている．[*2] インデントそのものもプログラム上で意味を持つため，ソースコード 4 を下のように書くとエラーになる．

[*2] この他にも Python にはいくつかのコーディング規約が定められている．詳しくは https://peps.python.org/pep-0020/ を参照．コーディング規約に従うに越したことはないが，初学の段階では過度に気にしなくても良い．自分の書いたプログラムを外部に公開するような際には気にしておいた方が良い．

ソースコード 5　インデントに起因してエラーとなる例

```
1 if x > 0 :
2 print("Pro")
3 else:
4 print("Con")
```

2.3　豊富なライブラリ

Python は豊富な**ライブラリ**を有している．ライブラリとは，よく使う機能がパッケージ化されたものである．ライブラリを活用することで，自力でプログラムを書く手間が大幅に削減される．Python を使ったプログラミングでは，プログラムを書き始める前に自分がやりたい処理に使えそうなライブラリがないかインターネットで検索することから始めるのが良い．特に数値計算では，自分でプログラムを書くよりもライブラリを利用した方が計算の高速化に繋がることが多い．[*3] ライブラリは積極的に活用しよう．

なお，全学計算機システムに導入されているソフトウェアは原則として授業で使用されるものに限られており，管理者権限が必要なソフトウェアをユーザがインストールすることはできない．ただし，`pip` コマンドからインストール可能な Python のパッケージ（ライブラリ）についてはユーザディレクトリにインストールして利用しても良いことになっている．インストールの際は，`pip install --user` **パッケージ名**を使うこと（全学計算機システムの Web サイト `https://www.u.tsukuba.ac.jp/ufaq/python1/` 参照）．

2.4　Python の予約語

表 1 の単語は，プログラム中で変数として使ってはいけない．これらは**予約語**と呼ばれ，変数として使用した場合は `SyntaxError` になる．

[*3] 数値計算のアルゴリズムそのものに対する理解を深めるといった目的であれば，一度は自力でプログラムを書いてみるのが一番勉強になる．

表 1　Python の予約語

and	class	except	global	None	return
as	continue	exec	if	nonlocal	True
assert	def	False	import	not	try
async	del	finally	in	or	while
await	elif	for	is	pass	with
break	else	from	lambda	raise	yield

2.5　PEP8

Python のコーディング規約として最も有名なのが **PEP8**（Python Enhanced Proposals 8）である．[*4] コーディング規約とは，コードを書く際の決まりごとであり，マナーのようなものである．初学の段階からコーディング規約を気にする必要はないが，将来的に自分の書いたコードを外部に公開するような場合には気にしておいた方が良い．

3　型

Python で扱うことができるのは，**数値**，**真偽値**，**文字列**の三種類である．これらの種類は**データ型**と呼ばれる．

3.1　数値型

Python の数値型には，**整数（int）**，**浮動小数点（float）**，**複素数（complex）**の三種類がある．文字通り整数は整数型であり，小数点を含む数字は型になる．Python では複素数も扱うことができる．虚数単位には i ではなく j を使うので注意．例えば，$1 + 2i$ は `1+2j` と書く．あるいは，虚数単位を書かずに，`complex(1,2)` と書くこともできる．複素数の実部は `real`，虚部は `imag` で取り出すことができる（ソースコード 6）．ここで，`real` や `imag` は複素数型の変数に紐づいた関数であり，このような関数のことを**メソッド**と呼ぶ．メソッドの使い方は，**変数名．メソッド名**である．なお，複素数はその実部や虚部を整数で書いても浮動小数点型として扱われる．

[*4] https://peps.python.org/pep-0008/

ソースコード 6　複素数の実部，虚部を取り出す

```
1 (1+2j).real
2 (1+2j).complex
```

これらの数値型に対する主要な演算については表 2 を参照．以下にいくつかの注意点を列挙する．

- 同じ型同士の四則演算の結果は原則としてその型になる．
- ただし，整数同士の割り算（/）の結果は整数型ではなく，浮動小数点型になる．
- 整数型と浮動小数点型の演算結果は浮動小数点型になる．
- int() や float() で囲めば，文字列も数値型へ変換可能．
- **浮動小数点型の数値は内部的にはその数値に最も近い近似値を扱っているため誤差を持つ．**

表 2　主な演算

演算	結果	注
a + b	a と b の和	
a - b	a と b の差	
a * b	a と b の積	
a / b	a と b の商	a, b が int 型でも結果は float 型
a // b	a と b の商を切り下げたもの	complex 型には使えない
a % b	a/b の剰余	complex 型には使えない
a ** b	a の b 乗	
pow(a, b)	a の b 乗	
abs(a)	a の絶対値	
int(a)	a を int 型に変換	a が float 型なら小数点以下切り捨て
float(a)	a を float 型に変換	
complex(a, b)	実部 a，虚部 b の複素数	a+bj と同じ
a.conjugate()	複素数 a の共役複素数	

3.2　真偽値

真偽値（bool）とは，真（True）か偽（False）の二値だけを取る型である．真偽値は

条件分岐などで用いられる．例えば，二つの数値を比較するとその結果が真偽値として与えられ，条件分岐やループの終了条件に用いられる．数値の比較には**比較演算子**を使う．代表的な比較演算子を表 3 にまとめておく．注意すべき点として，

- **浮動小数点数同士の等号比較は信頼できない．**

表 3　主な比較演算子

比較演算子	意味
`a < b`	a は b よりも小さい
`a <= b`	a は b 以下
`a > b`	a は b よりも大きい
`a >= b`	a は b 以上
`a == b`	a と b は等しい
`a != b`	a と b は等しくない

また，主要な `bool` 演算を表 4 にまとめておく．

表 4　主な `bool` 演算を表

演算	結果
`not x`	x が偽なら `True`，そうでなければ `False`
`x or y`	x が真なら x，そうでなければ y
`x and y`	x が偽なら x，そうでなければ y

4　代表的なコンテナ型

Python のデータ型には**コンテナ型**と呼ばれる複合的な型が存在する．よく使われるコンテナ型には，リスト，タプル，集合，辞書などがある．

4.1　リスト

Python の**リスト**は他のプログラミング言語における**配列**に相当するデータ型である．リストは，カンマで区切られた要素を鉤括弧で囲んで表現される．

ソースコード 7　リストの例

```
1 a = [0, 1, 2, 3, 4]
2 b = ["A", "B", "C", "D"]
3 c = [0, "ABC", 0.43, "xyz"]
```

例えば，ソースコード 7 において，a は整数値を要素に持つリストであり，b は文字列を要素に持つリストである．c のように異なるデータ型であっても同じリストに格納することができる．リストの各要素へのアクセスには**インデックス**と呼ばれる整数値を使う．Python のインデックスは 0 から始まる．例えば，a[0] と書くとリスト a の最初の要素である 0 にアクセスできる．代表的なリストに対する演算や処理を表 6 に挙げる．append はリスト型変数に紐づいたメソッドである．

表 5　リストに対する基本的な処理

演算	出力値	意味
a + b	リスト	二つのリストを結合し一つのリストを作る
len(a)	整数値	リスト a の長さ（要素数）を取得
"x" in a	bool 値	リスト a に指定した要素が含まれているか
a[0] = "x"	リスト	リスト a の特定の要素を置換
a.append("x")	リスト	リスト a の末尾に要素を追加

4.2　リストに対する処理

使用頻度の高いリストメソッドについては表 6 を見よ．ここでは aList がリストである．

表 6　よく使うリストメソッド

メソッド	使用方法	注
append	aList.append(item)	リストの末尾に新しい要素 item を追加
insert	aList.insert(i,item)	リストの i 番目の位置に新しい要素 item を挿入
pop	aList.pop()	リストの最後尾の要素を削除
pop	aList.pop(i)	リストの i 番目の位置の要素を削除
sort	aList.sort(key=None, reverse=False)	リストの要素を小さい値から大きい値の順番に並び替える． key にソートルールを決める関数を指定できる．reverse が True の時，リストは小さい値から大きい値へソートする．
reverse	aList.reverse()	リストを逆順にする
count	aList.count(item)	要素 item の個数を数える

4.3　タプル

Python における**タプル**はリストとよく似たオブジェクトであり，カンマで区切られた要素で表現される．

ソースコード 8　タプルの例

```
1 a = 0, 1, 2
2 b = (0, 1, 2)
```

タプルに括弧は不要であるが，上の例のように丸括弧で囲むこともできる．リスト同様に，タプルの要素へのアクセスはインデックスで行い，タプル同士の結合も可能である．リストとの違いは，一度作成されたタプルは修正できない，という点である．このような性質は**イミュータブル**と呼ばれる．タプルがよく使われる例として，複数の変数の初期化,

ソースコード 9　複数の変数の初期化

```
1 a, b = 1, 2
```

あるいは，変数の値の交換,

ソースコード 10　変数の値の交換

```
1 b, a = a, b
```

などが挙げられる．

4.4 集合

Python における**集合**は，数学における集合を表すオブジェクトである．集合はその要素を間まで区切り波括弧で囲むことで表現する．

ソースコード 11　集合の例

```
1 s = {0,1,2}
```

set という関数を使うことで，リストを集合に変換することができる．

ソースコード 12　リストを集合に変換

```
1 s = set([0, 1, 2])
```

集合に対する代表的な演算を表 7 に挙げる．

表 7　集合に対する代表的な演算

演算	出力値	意味
s1 | s2	集合	二つの集合から和集合を作る
s1 & s2	集合	二つの集合から積集合を作る
s1 ^ s2	集合	二つの集合の対称差

4.5 辞書

Python における**辞書**とは，**キー**と**バリュー**のペアからなるデータ型のことである．辞書では，「キー：バリュー」という要素からなり，キーを使って辞書の要素にアクセスする．イミュータブルなものであれば何でもキーに使うことができる．辞書の例として，以下を考えよう．

ソースコード 13　辞書の例

```
1 d = {"A":203, "B":921, "C":-271}
```

例えば，print(d["A"]) と書くと 203 が出力される．また，上の辞書 d に対して，

ソースコード 14　辞書に要素を追加

```
1 d["D"] = 334
```

とすると辞書 d に新たな要素"D":334 が追加される．リストと同様に，辞書のバリューは変更可能である．辞書はファイル操作やデータ処理などで有用である．

5 関数

5.1 def 文

特定の処理を**関数**という形で定義することで，その処理を行うコードを毎回書く必要がなくなる．関数を定義する場合は，以下のように def 文を使ったコードブロックを作ればよい．

ソースコード 15　関数を定義する

```
1 def func(x, y):
2     ans = x + y
3     return ans
```

ソースコード 15 では，func という名前の関数が定義され，x と y を func に渡すとその和 ans が返ってくる．x と y のように関数に入力される変数のことを**引数**と呼び，ans のように関数から返ってくるもののことを**戻り値**と呼ぶ．

関数の引数にはデフォルトの値を設定することも可能である．以下のソースコード 16 を見てみよう．

ソースコード 16　デフォルト値つきの関数を定義する

```
1 def func(x, y=2, z=3):
2     ans = x + y + z
3     return ans
```

デフォルト値が設定されている引数 y と z については，これらを変更したい時だけ引数を指定すれば良い．ソースコード 16 の関数 func は以下のように使うことができる．

ソースコード 17　デフォルト値つきの関数の使い方

```
1 a = func(1) # a=1+2+3
2 a = func(1,4,5) # a=1+4+5
3 a = func(1,z=0) # a=1+2+0
4 a = func(1,z=2,y=3) # a=1+3+2
```

5.2 ラムダ式

なお，関数を定義する別の方法として，**ラムダ式**を使う方法がある．ラムダ式は `def` 文を使うほどではない単純な処理を定義したい場合に有用である．例えば，ソースコード 16 をラムダ式を使って書くと，

ソースコード 18　ラムダ式の使い方

```
1 func = lambda x, y=2, z=3: x+y+z
```

となる．定義した関数の使い方は `def` 文を使った場合と全く同様である．

6 データの入出力

数値計算を行う際，得られた結果を出力データとして保存しておくことは極めて大切である．結果の妥当性などを考察するためには, 出力データの可視化や解析が不可欠である．また，同じ問題を違うパラメタセット上で計算したい場合，プログラムに渡す入力パラメタを入力データとして保存しておくことも有益である．毎回プログラムを書き換える必要が省けるだけでなく，計算に使ったパラメタの情報を入力ファイルとして保存しておくこともできる．あるプログラムの実行結果として得られた出力データを，別のプログラムへの入力データにする，というのもよくある使い方の一つである．これらは全て，数値計算によって得られた結果の再現性を担保することに通ずる．

Python でファイルを開くためには，`open` 関数を用いる．ファイルを開く時には，そのファイルの用途（読み込み専用，書き込み専用，追記用など）とデータ形式（テキストデータ，バイナリデータ）を選択する必要がある．用途を指定せずにファイルを開くと，読み込み専用となる．データ形式を指定しない場合，テキストデータが自動的に選択される．これらの用途，データ形式の選択は，`open` 関数のオプションから行う．表 8 にオプションをまとめておく．

表 8　open 関数のオプション

オプション	意味
r	読み込み専用（デフォルト）
w	書き込み専用
a	追記用
t	テキストモード（デフォルト）
b	バイナリモード

例えば，filename という名前のファイルをテキストデータかつ読み込み専用として開く場合は，以下のように open 関数を使えば良い．open 関数は，ファイルオブジェクトを返す関数である．下の例では，f にファイルオブジェクトが代入されている．

ソースコード 19　テキストデータかつ読み込み専用で開く

```
1 f = open("filename")
```

また，以下のように，テキストデータかつ読み込み専用であることをオプションで明示的に書いても良い．

ソースコード 20　テキストデータかつ読み込み専用で開く

```
1 f = open("filename","rt")
```

テキストモードかつ書き込み用で開く場合には以下のようになる．

ソースコード 21　テキストデータかつ書き込み専用で開く

```
1 f = open("filename","w")
```

一度 open 関数でファイルを開いた後は，ファイルオブジェクト f を通してデータの読み書きを行うことになる．

7　ライブラリを import する方法

Python でパッケージやモジュールを呼び出す場合には import 文を書く．import 文の書き方は一意ではなく，多様な書き方が存在する．例えば，NumPy [*5] を呼び出す場合，最も素朴な import 文の書き方は，

[*5] https://numpy.org/ja/

ソースコード 22　NumPy を呼び出す方法

```
1 import numpy
```

である．この場合，以下で NumPy の関数を使う際には numpy．**関数名**（）と書く必要がある．

よく使うパッケージやモジュールを import する際，慣例的な略称をつけることも多い．例えば，NumPy の場合だと，

ソースコード 23　NumPy を呼び出す方法

```
1 import numpy as np
```

という書き方がよく使われる．この場合，以下で NumPy の関数を使う際には np．**関数名**（）と書く必要がある．

パッケージの中の特定のモジュールだけを import する場合には，

ソースコード 24　NumPy を呼び出す方法

```
1 from numpy import random
```

と書くことができる．ソースコード 27 では，例として random モジュールを import した．以下で random モジュールの関数を使う際には，random．**関数名**（）と書けばよい．

8　import によるスクリプトの分割

import を使うことで, すべての処理を一つの長大なスクリプトに記述することを避け, 機能ごとに複数のスクリプトに分割して保存することができる．スクリプトを適切に分割することで，プログラムの構成が分かりやすくなり，コードの修正や再利用も容易になる．[*6]

ここでは，簡単な例を用いて import によるスクリプトの分割方法を説明する．まず，以下のような関数を myfunc.py という名前のファイルに保存する．

ソースコード 25　myfunc.py に保存する関数

```
1 def square(x):
2     return x**2
3
```

[*6] 初学者のうちはあまり意識する必要はないが，import を用いたスクリプトの分割は，実践的なプログラミングにおいて不可欠なスキルである．

```
4 def cube(x):
5     return x**3
```

次に，myfunc.py と同じディレクトリに，以下のスクリプトを main.py として保存する．

ソースコード 26　myfunc.py を読み込むスクリプト

```
1 import myfunc
2
3 x = 2
4
5 y = myfunc.square(x)
6 z = myfunc.cube(x)
7
8 print(y)
9 print(z)
```

main.py の 1 行目で，myfunc.py に記述された内容を読み込んでいる．このとき，ファイル名の拡張子.py は書かないことに注意．myfunc.py で定義した関数を使用するには，main.py の 5 行目や 6 行目のように，**ファイル名**.**関数名**の形で記述する．このように，よく使用する関数を別のスクリプトにまとめておき，必要なときに import することで，メインのスクリプトを簡潔に保つことができる．また，特定の関数だけを読み込みたい場合には，以下のソースコード 27 のように記述することもできる．この場合，square を直接読み込んでいるため，myfunc.square(x) ではなく square(x) と記述できる．

ソースコード 27　特定の関数のみを読み込む方法

```
1 from myfunc import square
2
3 x = 2
4 y = square(x)
5
6 print(y)
```

9　matplotlib による作図

9.1　最も基本的な使い方

matplotlib は非常に便利なグラフ描画のためのライブラリである．[*7] matplotlib の最も基本的な使い方をソースコード 28 に示す．ここでは，Python で数値計算を行う際に広く使われている NumPy ライブラリと併用している．`plt.plot()` は指定したデータをもとに，点や線を用いてグラフを描画するための関数である．基本的な使い方は以下の通りである．

```
plt.plot(x, y, fmt)
```

ここで，引数の順番には意味があり，それぞれ以下の役割を持つ．

- `x`: 横軸のデータ
- `y`: 縦軸のデータ
- `fmt`: 線の種類, データ点の形状, および色を指定する文字列

ソースコード 28　matplotlib の最も基本的な使い方

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 x = np.arange(-np.pi, np.pi, 0.1)
5 y = np.sin(x)
6
7 plt.plot(x, y, "-ob")
8
9 plt.savefig("sinx.pdf")
```

ソースコード 28 の出力結果を図 1 に示す．ソースコード 28 の 7 行目の「`-ob`」が `fmt` の具体定な書き方である．「`-`」は実線,「`o`」は `o` の形のマーカー,「`b`」は青色を意味する．線の種類，マーカーの種類，色を指定する順番は任意である．つまり,「`o-b`」や「`bo-`」のように書いても良い．いくつかの代表的な線，マーカー，色の指定方法をそれぞれ表 9, 10，11 に示す．

[*7] https://matplotlib.org/stable/gallery/index

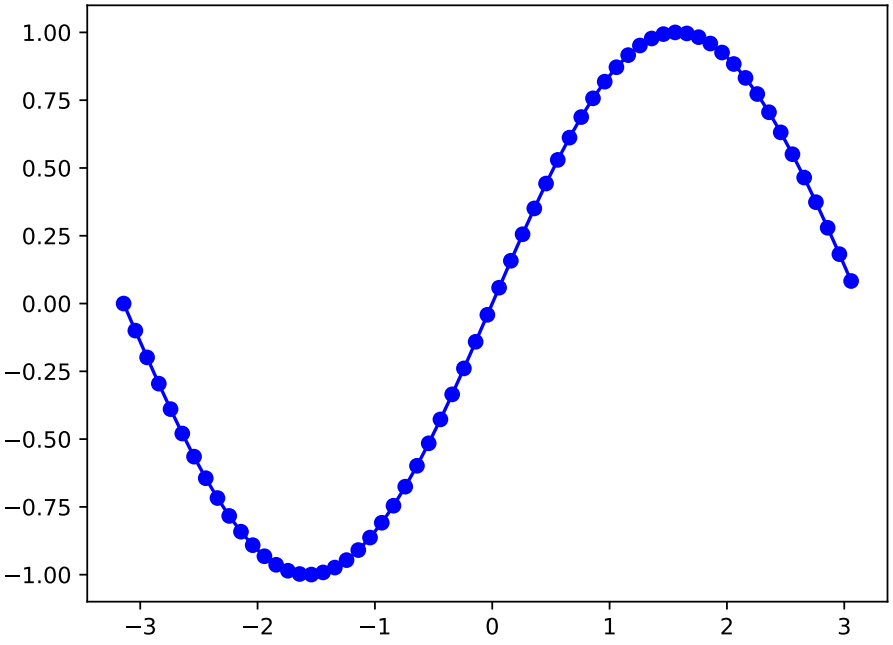


図 1　ソースコード 28 の出力結果

表 9　主な線の種類

スクリプト上での記号	線の種類
-	実線
--	破線
-.	一点鎖線
:	点線

表 10　主なマーカーの種類

スクリプト上での記号	線の種類
.	点
,	ピクセル
o	丸
^	三角
v	下三角
<	左三角
>	右三角
s	四角
d	菱形
+	＋
x	×

表 11　主な色の種類

スクリプト上での記号	色
r	赤
g	緑
b	青
c	シアン
m	マゼンタ
y	黄
k	黒
w	白

9.2　誤差バー付きのグラフ

数値計算や実験で得られるデータには，統計的な誤差が含まれることがある．このようなデータを可視化する際には，各データ点に誤差の大きさを表す「誤差バー (error bar)」を付けることが多い．matplotlib では, `plt.errorbar()` を使うことで，誤差バー付きのグラフを作成できる．基本的な使い方は以下の通り．

```
plt.errorbar(x, y, yerr=yerr, fmt="o")
```

plt.errorbar() の主な引数は以下の通りである.

- x: 横軸のデータ
- y: 縦軸のデータ
- yerr: 縦方向の誤差の大きさ
- xerr: 横方向の誤差の大きさ
- fmt: データ点の形状や線の種類，および色を指定する文字列で, fmt="o"のように書く
- color: データ点や誤差バーの色で，color="b"のように書く
- capsize: 誤差バーの両端に付ける横線の長さを指定し，capsize=3 のように書く

yerr を指定した場合，各データ点を中心として上下に誤差バーが表示される．xerr を指定すれば，横方向にも誤差バーを付けることができる．以下に簡単な例を示す．ソースコード 29 で，x と y はそれぞれデータ点の横軸と縦軸の値であり，yerr は各データ点の縦方向の誤差を表す.

ソースコード 29　誤差バー付きのグラフ

```
1  import matplotlib.pyplot as plt
2  import numpy as np
3
4  x = np.array([1, 2, 3, 4, 5])
5  y = np.array([2.1, 3.9, 6.2, 7.8, 10.1])
6  yerr = np.array([0.3, 0.5, 0.4, 0.6, 0.5])
7
8  plt.errorbar(x, y, yerr=yerr, fmt="o", color="b", capsize=3)
9
10 plt.xlabel("x")
11 plt.ylabel("y")
12
13 plt.savefig("errorbar.pdf")
```

ソースコード 29 の出力結果を図 2 に示す.

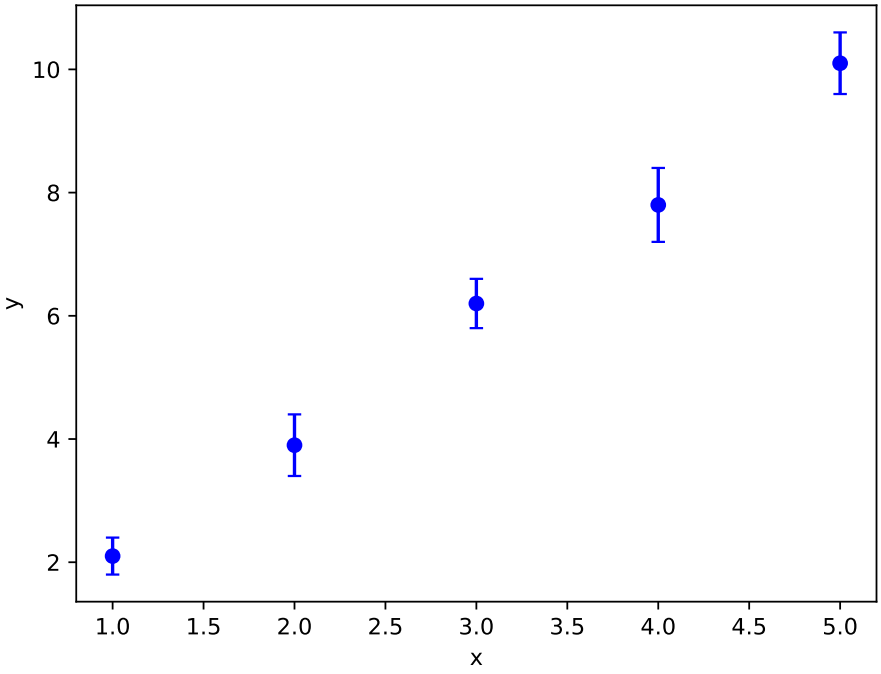


図 2　ソースコード 29 の出力結果

9.3　散布図

2 つの変数の関係を調べる際には，「散布図 (scatter plot)」がよく用いられる．散布図では，各データを点として表示し，変数間の相関やデータの分布を視覚的に確認できる．matplotlib では，`plt.scatter()` を使って散布図を作成できる．基本的な使い方は以下の通り．

```
plt.scatter(x, y, color="b", s=50)
```

主な引数の役割は以下の通り．

- `x`: 横軸のデータ
- `y`: 縦軸のデータ
- `color`: データ点の色で, `color="b"`のように指定する
- `s`: データ点の大きさであり，`s=50` のように指定する
- `marker`: データ点の形状であり，`marker="o"`のように指定する

- `label`: データに付ける名前であり，凡例を表示する際に使用する．

ソースコード 30 を見てみよう．ソースコード 30 の出力結果は図 3 のようになる．

ソースコード 30　散布図の作成

```
1  import matplotlib.pyplot as plt
2  import numpy as np
3
4  x = np.array([1, 2, 3, 4, 5, 6])
5  y = np.array([2.1, 3.8, 5.2, 4.9, 7.1, 8.3])
6
7  plt.scatter(x, y, color="b", s=50)
8
9  plt.xlabel("x")
10 plt.ylabel("y")
11
12 plt.savefig("scatter.pdf")
```

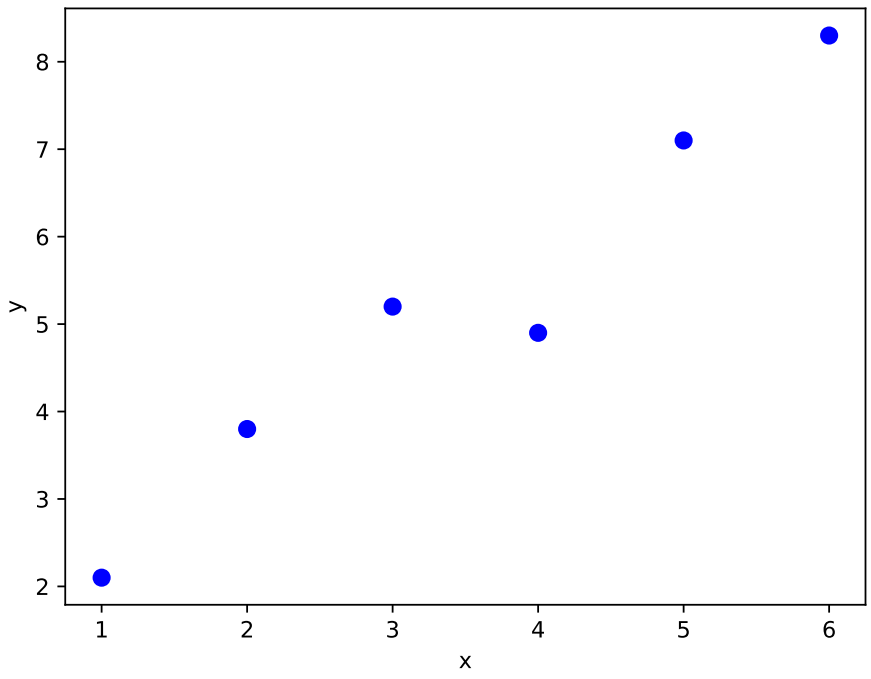


図 3　ソースコード 30 の出力結果

なお，plt.plot(x, y, "o") を使っても同様にデータ点のみを表示できる．しかし，plt.scatter() は散布図の作成に特化したメソッドであり，各データ点の色や大きさを個別に指定できるという特徴がある．状況に合わせて使い分けてみてほしい．

9.4 複数のデータのプロット

複数のデータを同じグラフ上に表示することで，それぞれの結果を比較することができる．matplotlib では，描画メソッドを複数回呼び出すことで，同じグラフ上に複数のデータをプロットできる．

以下では，$\sin x$ と $\cos x$ を同じグラフ上に表示する例を示す．

ソースコード 31　複数のデータのプロット

```
1  import matplotlib.pyplot as plt
2  import numpy as np
3
4  x = np.arange(-np.pi, np.pi, 0.1)
5
6  y1 = np.sin(x)
7  y2 = np.cos(x)
8
9  plt.plot(x, y1, "-b", label="sin(x)")
10 plt.plot(x, y2, "--r", label="cos(x)")
11
12 plt.xlabel("x")
13 plt.ylabel("y")
14 plt.legend()
15
16 plt.savefig("sin_cos.pdf")
```

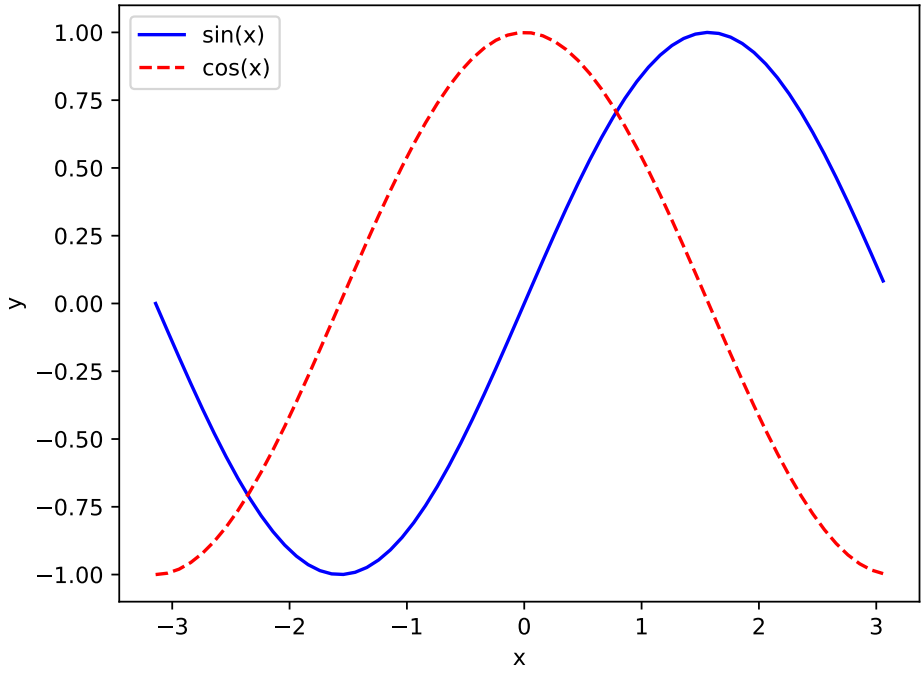


図 4　ソースコード 31 の出力結果

この例では，`plt.plot()` を 2 回呼び出し，それぞれのデータを異なる線種と色で表示している．`"-b"`で青色の実線，`"--r"`で赤色の破線を指定している．

複数のデータを同じグラフ上に表示する際には，凡例を付けることで各データを区別しやすくなる．`label` は各データに付ける名前を指定するための引数であり，実践的な作図では頻繁に使うことになる．`plt.legend()` を呼び出すことで，`label` で指定した名前が凡例として表示される（ソースコード 31 の 14 行目のように `plt.legend()` を呼び出さないと `label` で指定した名前が凡例として出力されないので注意）．

なお，`plt.plot()` だけでなく，`plt.errorbar()` や `plt.scatter()` も複数回呼び出すことができる．したがって，誤差バー付きのデータと曲線を同じグラフ上に表示するといったことも可能である．

9.5　LaTeX の記法を使う

なお，matplotlib による作図では LaTeX を使うことができる．ソースコード 28 に対して，LaTeX の記法に従って軸のラベルを入れるようにしたのがソースコード 32 である．

r"..."と書くことにより，その中で LaTeX の記法を使うことができる.[*8] ソースコード 32 の出力結果を図 5 に示す.

ソースコード 32　matplotlib で LaTeX の記法を使う

```
1  import matplotlib.pyplot as plt
2  import numpy as np
3
4  plt.rc('text', usetex=True)
5  plt.rc('font', family='serif')
6
7  x = np.arange(-np.pi, np.pi, 0.1)
8  y = np.sin(x)
9
10 plt.title(r"Graph Title")
11 plt.xlabel(r"$x$")
12 plt.ylabel(r"$\sin x$")
13 plt.plot(x, y, "-ob")
14
15 plt.savefig("sinx_latex.pdf")
```

[*8] r"..."の r は raw string に由来し，バックスラッシュなどの特殊文字を生の文字列として取り扱う.

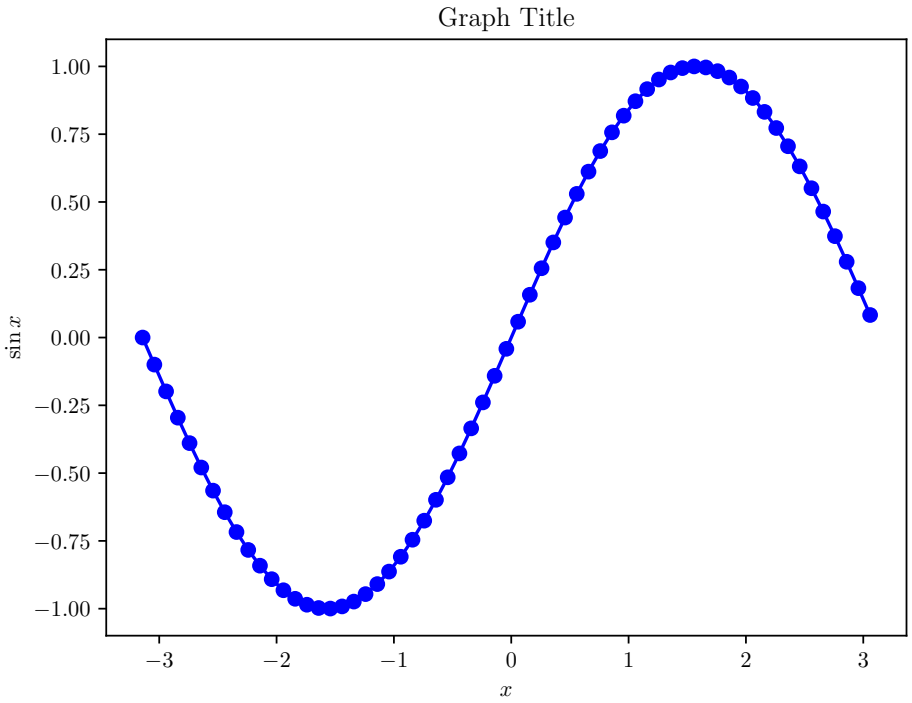


図 5　ソースコード 32 の出力結果

9.6　変数の埋め込み

さらに，`f"..."`を使うことで，Python の変数を埋め込むことができる．[*9] 先ほどの`r"..."`と組み合わせて`rf"..."`と書けば，LaTeX 記法の使用と変数埋め込みの両方に対応できる．ソースコード 33 とその出力結果 6 を見て欲しい．

ソースコード 33　matplotlib での Python 変数の埋め込み

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 plt.rc('text', usetex=True)
5 plt.rc('font', family='serif')
6
7 a = 2
```

[*9] `f"..."`の `f` は formatted strings literals に由来し，Python の `format` メソッドを使いやすくしたもの．

```
 8 b = -1
 9 c = -5
10 d = 3
11
12 x = np.arange(-np.pi, np.pi, 0.1)
13 y = a*x*x*x + b*x*x +c*x + d
14
15 plt.title(rf"$a = {a}, b = {b}, c = {c}, d = {d}$")
16 plt.xlabel(rf"$x$")
17 plt.ylabel(rf"$ax^3+bx^2+cx+d$")
18 plt.plot(x, y, "-ob")
19
20 plt.savefig("poly_abcd.pdf")
```

Python 変数の埋め込みを応用して, 出力ファイル名に入力パラメタの情報を埋め込むこともできる．例えば, ソースコード 33 の 20 行目を `plt.savefig(f"poly_a{a}b{b}c{c}d{d}.pdf")` と変更すれば,「`poly_a2b-1c-5d3.pdf`」という名前で図が保存される．入力パラメタに応じて出力ファイル名が自動的に変わるため，不用意な図の上書きを防止できるだろう．

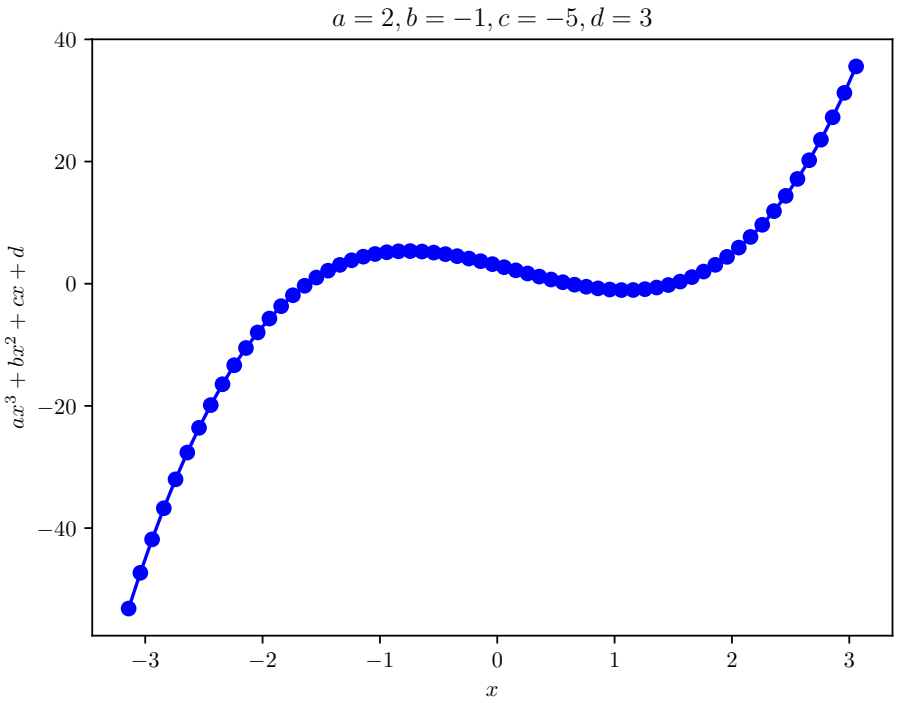


図 6　ソースコード 33 の出力結果

10　仮想環境

Python では，プロジェクトごとに独立した実行環境を作成することができる．このような環境を**仮想環境（virtual environment）**と呼ぶ．仮想環境を利用すると，プロジェクトごとに使用するパッケージやそのバージョンを個別に管理することができる．例えば，あるプログラムでは古いバージョンの Numpy が必要である一方，別のプログラムでは新しいバージョンが必要になる，といった場合があり得る．仮想環境を利用すれば，それぞれの環境に異なるバージョンの NumPy をインストールできるため，互いに影響を与えることなくプログラムを実行できる．

Python には，仮想環境を作成するための `venv` という機能が標準で用意されている．例えば，以下のコマンドを実行すると，`myenv` という名前の仮想環境を作成できる．

```
1 python3 -m venv myenv
```

Linux や macOS では，作成した仮想環境を以下のように有効化する．

```
source myenv/bin/activate
```

仮想環境が有効な状態で，

```
pip install numpy matplotlib
```

などを実行すると，これらのパッケージは現在の仮想環境にインストールされる．仮想環境の使用を終了する場合には，

```
deactivate
```

を実行する．

初学者のうちは仮想環境を強く意識する必要はないが，複数のプログラムを開発したり，長期間にわたって研究や開発を行ったりする場合には，仮想環境を用いて実行環境を管理することが重要になる．